

WaveVis inverse-sheet architecture for Moana-ocean breaking-wave linkage arrays

Alan N. Pham

AO Labs, WaveVis simulator, inverse deformation sheet record

WaveVis is an inverse simulator for a mostly flat rectangular linkage sheet with one localized plunging-wave deformation rising out of the sheet. The primary visual target is the Moana ocean reference geometry supplied for this project: a translucent, water-like sheet rising from a flat surface, swelling into a rounded crest, curling forward and downward, and returning to flat without walls, cavities, or extruded tunnel behavior. The intended artifact is not an extruded ribbon, half-pipe, swept cross-section, roof, bridge, or hollow shell. It is a full array of linkage cells over a fixed flat perimeter: the center region ramps upward, forms a rounded crest, projects a lip forward and downward, and fades smoothly back into the surrounding sheet in all directions. The reference geometry is an acceptance target, not a success claim. A generated state is still open if the human-facing render reads as a tunnel, wall-covered cavity, angular blob, missing curl, or broken linkage even when code checks pass. This paper replaces the earlier narrow constraint-proof PDF as the visible system record. It defines the reference geometry, coordinate systems, profile editors, slider semantics, topology invariants, rendering views, failure modes, and verification gates required before WaveVis work can be called correct.

1 Introduction

WaveVis exists to explore whether a cellular Sarrus-style linkage array can approximate visually meaningful three-dimensional sheet deformations while preserving mechanical readability. The simulator therefore has two simultaneous jobs. First, it must generate a target shape that resembles the supplied Moana ocean references rather than a generic height field. Second, it must keep the linkage graph legible: cells, nodes, connector pairs, and edges must remain visible and connected rather than being hidden under cosmetic surfaces or erased when the shape becomes difficult.

The current design problem is stricter than drawing a pretty profile. A side silhouette can look acceptable while the actual three-dimensional sheet has a wall across the tube, a cavity where the lip should be open, an extruded-barrel cross-section, or zig-zagging linkage legs that no longer read as a mechanism. Conversely, a mesh can satisfy a local numerical check while failing the human outcome: a full rectangular sheet with a localized plunging wave. WaveVis must therefore treat the visible artifact, the mechanism topology, the parameter semantics, and the regression checks as one architecture.

The central correction captured here is that a breaking wave is not made by sweeping one two-dimensional profile sideways. A swept profile creates a tunnel, canopy, or half-pipe. The required object is a localized deformation field on a flat sheet:

$$\Phi : (x, y) \mapsto (x + \Delta_x(x, y), y + \Delta_y(x, y), \Delta_z(x, y)),$$

where the displacement is concentrated near the middle span, gradually fades toward the lateral sides, and is exactly zero on the perimeter. The curl must be strongest near the centerline and weaker toward the sides. This spatial falloff is part of the target shape, not a rendering trick.

1.1 Reference geometry

The design reference is the Moana ocean: a smooth, localized volume of water that rises out of a larger flat body, grows into a broad rounded crest, and curls into a downward lip while leaving an open underside. These reference frames are not decorative. They are the visual contract for the simulator's target geometry. A successful WaveVis state should read as a simplified linkage approximation of this water form: the outer radius is large and smooth, the inner curl radius is smaller, the lip projects forward before curling down, and the side field fades away instead of forming a constant barrel. This is

deliberately stricter than matching a side-profile curve: the full linkage array in side and isometric views must preserve the open curl without side walls, erratic zig-zags, disconnected cells, or hidden filler geometry.

Current verification boundary. This document records the target, architecture, and required gates. It does not by itself prove that the current generator has matched the supplied references. The rendered full-array artifact remains the source of truth. If the page, screenshots, or live route do not visibly match the Moana-like localized plunging wave while preserving linkage cells and flat perimeter, the correct state is “open visual mismatch,” not “reference matched.”



Figure 1. Primary supplied reference geometry. The target form is a localized plunging water sheet with a rounded crest, open underside, and forward/downward lip. The WaveVis sheet must approximate this geometry while preserving flat perimeter boundary nodes and visible linkage cells.



(a) Open tube and suspended lip.



(b) Forward/downward curl at the water edge.



(c) Localized mound emerging from flat water.



(d) Rounded column without a hard perimeter wall.

Figure 2. Additional supplied reference frames. Together they define the acceptance target: water-like surface continuity, strong center curl, smooth lateral fade, no side wall covering the underside, and no swept tunnel extrusion.

2 Target outcome

2.1 Human-facing shape

The accepted target is a “flat sheet plus localized plunging wave”:

- the outer perimeter of the rectangular sheet remains flat and fixed;
- the wave emerges through a broad, smooth ramp rather than a vertical wall;
- the crest is rounded, with no top dent;
- the lip projects forward and curls downward;
- the underside or tube is visible from side view, not covered by a side wall;
- the shape fades back to flat in every direction;
- the array remains a full linkage field with nodes and legs, not a hidden or clipped shell.

Figure 3 gives the practical acceptance boundary. The shape must read as a local wave in a sheet, not as an extrusion. A single cross-section can guide the profile, but the generated three-dimensional field must vary across span.

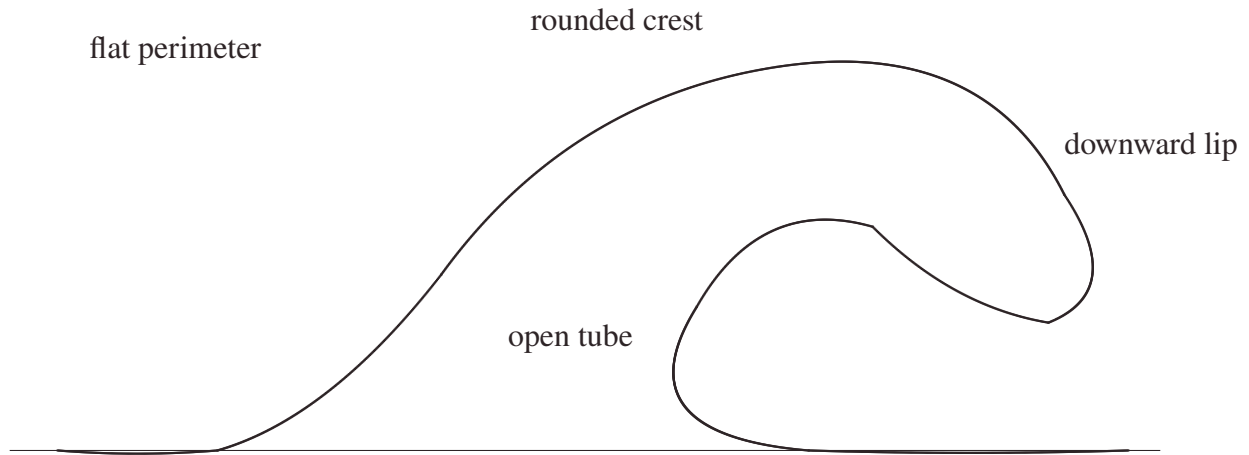


Figure 3. Desired side silhouette as an outcome object. The drawing is not a literal mesh recipe. It is the visible side-view contract: smooth back ramp, larger outer radius, smaller inner radius, curled lip, and flat return. The three-dimensional solver must then localize this silhouette in the middle of a full sheet rather than sweep it as a constant tunnel.

2.2 Hard constraints

The WaveVis generator and renderer must preserve the following hard constraints:

1. **Flat perimeter.** Boundary nodes must remain on the rest sheet. No side, front, or rear perimeter should be pulled into the wave.
2. **Full array visibility.** The rendered artifact must show the actual linkage grid. Hiding or ghosting broken cells does not count as a fix.
3. **Connector closure.** A cell connector is an actual shared graph location. Lines should meet at nodes; apparent detached legs are a failure.
4. **No erratic legs.** Long arms, spikes, zig-zags, random backtracking, and visible overlaps indicate a broken geometry or render path.
5. **Smooth strain sharing.** Adjacent rows and columns should change gradually enough that the mesh reads as a continuous mechanism field.

6. **Localized wave.** The curl must be strongest near the center and fade laterally. A constant side-to-side tunnel is not the target.

7. **Side/cross-section separation.** Side view shows the side view of the full three-dimensional render. Cross-section view is the only mode that intentionally slices or projects the center profile.

8. **No filler geometry.** Cosmetic walls, caps, planks, bridges, hidden couplers, or masks must not be used to conceal a bad mechanism.

3 Coordinate model

The simulator starts from a rectangular rest lattice. A node with row i and column j has rest position

$$p_{ij}^0 = (x_j, y_i, 0).$$

The target is a displacement field

$$p_{ij}^* = p_{ij}^0 + m D(x_j, y_i; \theta),$$

where $m \in [0, 1]$ is the morph amount and θ represents all shape parameters except rigid placement.

The morph parameter only interpolates between rest and the finalized target. It must not change the target shape definition, remesh the lattice, or introduce a different topology.

The longitudinal coordinate $u \in [0, 1]$ measures progress through the wave region from rear/root to front/lip. The span coordinate $v \in [-1, 1]$ measures lateral distance from the centerline. A good WaveVis target uses both:

$$D(x, y) = A(u, v) P(u) + B(u, v) Q(u),$$

where $P(u)$ is the side-profile contribution, $Q(u)$ can include forward curl or secondary displacement terms, and A, B are smooth two-dimensional envelopes that go to zero at the sheet perimeter.

The key architectural rule is that $A(u, v)$ is not constant in v . A constant span envelope turns any attractive profile into a tunnel. A localized plunging wave requires $A(u, 0)$ large near the centerline and $A(u, \pm 1) \approx 0$ near the sides, with a monotone or smoothly unimodal falloff rather than a hard wall.

4 Generation pipeline

99

The target-building pipeline is:

100

1. sanitize rows, columns, profile mode, sliders, display flags, and URL parameters;

101

2. build the full rectangular node grid;

102

3. compute the canonical wave displacement from the selected profile mode;

103

4. apply the span envelope that localizes the wave in y;

104

5. preserve the boundary by forcing perimeter displacement to zero;

105

6. apply morph interpolation from rest to target;

106

7. apply rigid steer and position transforms after the shape is finalized;

107

8. build full rectangular edge and quad topology;

108

9. compute strain, bend, shear, dihedral, and display metrics;

109

10. render the full array, side view, or cross-section without substituting fake surfaces.

110

The distinction between shape parameters and rigid placement parameters is essential. Height, scale,

111

and the profile editor change the canonical target. Steer rotates the finished target around the anchor.

112

Position translates the finished target horizontally. Neither steer nor position may feed back into profile

113

curvature, curl, width, strain generation, or cell topology.

114

Stage	Contract	Failure caught
Target profile	Defines the desired side silhouette.	Blob, neck/head, hill without lip, top dent.
Span envelope	Localizes the wave in the sheet.	Swept barrel, side wall, U-shaped tube blocker.
Boundary clamp	Keeps the perimeter flat.	Pulled sheet edges, non-flat perimeter.
Morph	Interpolates rest to target only.	Half-morph mangling, topology change while sliding.
Topology	Keeps full node/edge/quad graph.	Missing grid, hidden cells, detached legs.
Renderer	Shows the actual mechanism.	Fake shell, hidden wall, side view as cross-section.

Table 1. Pipeline stages and the failure modes each stage must prevent.

5 Profile and slice editors

The custom xz editor is a side-profile reference. Its points are not cell locations. They are control points for a smooth reference curve that the surface generator samples. The final cell grid is still determined by the row and column count.

The custom xy slice editor is a span-envelope reference at a selected height or profile level. It should not define the ground footprint. Its purpose is to describe how wide the wave is at a slice through the deformation, particularly near the underside of the curl where unwanted walls can otherwise cover the tube. The slice marker on the xz editor identifies the height/profile level that the xy editor is editing.

Mathematically, the xz editor contributes a curve

$$P(u) = (x_p(u), z_p(u)),$$

and the xy editor contributes an envelope sample

$$W(u, v) \in [0, 1].$$

The actual target displacement is produced by multiplying these sources by a smooth support field. The editor curves should never be treated as a hard polygon whose boundary is stitched directly to the ground, because that creates walls, cavities, and straight connector chutes.

6 Slider semantics

128

Control	Intended effect	Must not do
morph	Blend rest sheet into the completed target.	Remesh, flip cells, create zig-zags, change profile identity.
scale	Uniformly enlarge or reduce the profile reference.	Change steer, position, or topology.
height	Vertical size of generated-mode wave.	Create top dents or side walls.
overhang	Forward reach of generated-mode wave.	Pull the lip into a straight vertical chute.
lipDip	Increase forward-and-down curl near the center.	Lower a terminal endpoint only, make a global hill, or form a swept tunnel.
lipSharpness	Tighten the terminal lip radius.	Explode spikes or change the broad wave body.
width	Scale the lateral active span.	Change centerline profile or create row popping.
groundTransition	Broaden the smooth ramp into the flat sheet.	Break the perimeter or erase the lip.
wallSmoothness	Round lateral side transitions.	Produce side walls covering the curl.
flatContribution	Share deformation with the surrounding apron.	Flatten the main wave.
steer	Rigid yaw rotation of the finished wave.	Bend, stretch, or reshape the wave.
position	Rigid horizontal translation of the finished wave.	Change silhouette, strain field, or curl.

Table 2. Control contract. Each slider has a narrow semantic role; cross-coupling that changes another slider's job is a bug unless the interaction is explicitly documented and verified.

7 Breaking-wave geometry

129

The desired generated-mode profile is closer to a localized plunging wave than to a cone, cylinder, or arch. The useful approximation is a tapered, asymmetric surface whose highest crest sits behind the curl, whose outer radius is larger than the inner radius, and whose underside remains open. Along the

130

131

132

centerline, a breaking wave profile can be represented as three joined curve families:

$$P(u) = \begin{cases} P_{\text{ramp}}(u), & 0 \leq u < u_c, \\ P_{\text{crest}}(u), & u_c \leq u < u_l, \\ P_{\text{lip}}(u), & u_l \leq u \leq 1. \end{cases}$$

The ramp rises from the sheet. The crest rounds over. The lip turns forward and downward. The derivative at the terminal lip should point downward when $\text{lipDip} > 90^\circ$:

$$\left. \frac{dz}{dx} \right|_{\text{tip}} < 0.$$

The final point is allowed to sit behind the farthest horizontal reach. In a real breaking wave, the maximum reach can occur at the shoulder while the lower tip tucks under the crest. Treating the farthest-right point as the tip is the mistake that produces a straight falling chute.

The three-dimensional field must multiply this centerline motion by a span falloff:

$$E(v; u) = \exp \left[- \left(\frac{|v|}{w(u)} \right)^\alpha \right],$$

with $w(u)$ varying through the wave. The curl should have a smaller active width than the broad back ramp, so the lip is localized and the tube remains visible from the side.

8 Mechanism topology

WaveVis uses a rectangular graph of nodes, horizontal profile edges, vertical span edges, and quads. Each interior node participates in the lattice; each cell is bordered by edges; and each connector must visually close. The graph is not allowed to become a set of isolated rows.

The mechanism evidence from the earlier proof remains relevant. The all-four-equal X-cell branch was rejected because the finite connector assignment does not close cleanly under the current overhang contract. The accepted branch is pairwise: east-west legs form one equal collinear pair, north-south legs form another equal collinear pair, and the angle between the pairs is allowed to vary. This pairwise rule preserves a readable linkage without forcing impossible equal-arm closure (*I*).

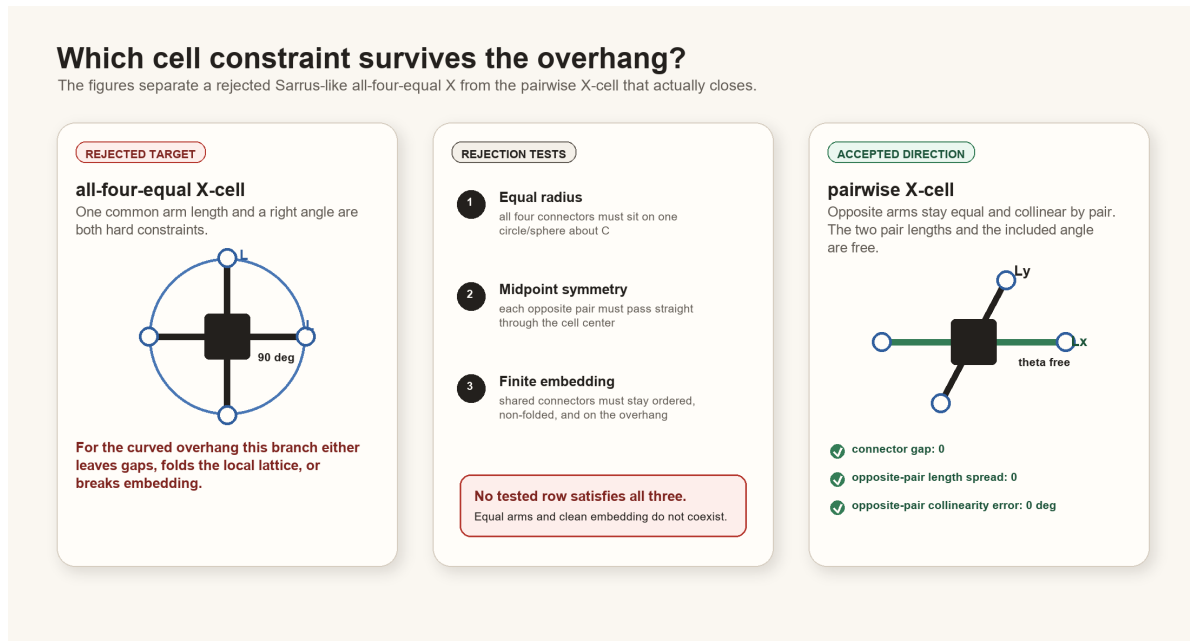


Figure 4. Mechanism contract inherited from the constraint proof. WaveVis should preserve exact shared contact and pairwise equal, collinear opposite arms rather than hiding infeasible all-four-equal geometry behind filler surfaces.

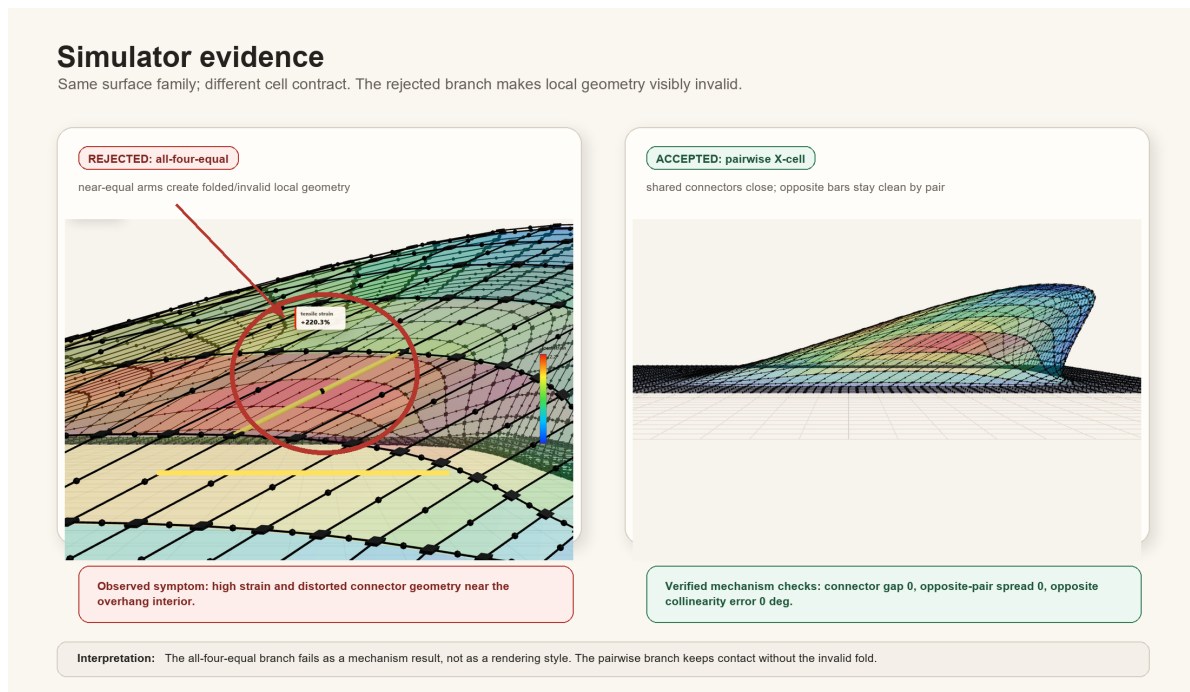


Figure 5. Simulator evidence for the linkage direction. The current architecture must keep this mechanism truth visible while the inverse target shape is improved.

9 Rendering modes

WaveVis has at least three distinct views:

- **3D/isometric view** shows the full rectangular array in perspective.
- **Side view** is a camera view of the full three-dimensional render from the side. It is not a cross-section and must not slice away rows.
- **Cross-section view** intentionally shows a selected slice or profile. This is the only mode that may look like a profile projection.

The view contract matters because a side view can hide the curl if lateral geometry forms a wall over the tube. In that case the correct fix is to reshape the three-dimensional surface so the lateral envelope follows the wave contour and fades out, not to switch the side view into a cross-section or hide the wall.

10 Verification gates

The generator should be checked with both numeric and visual gates. Numeric checks are not sufficient, but they catch repeated hard failures before visual review.

Gate	Measurement	Acceptance intent
Perimeter flatness	$\max_{\partial\Omega} \ p^* - p^0\ $.	Boundary displacement is zero or visually negligible.
Topology completeness	Expected node, edge, and quad counts.	Full rectangular sheet graph remains visible.
Connector closure	Edge endpoints meet their node coordinates.	No detached legs or fake bridges.
Lip curl	Crest height minus lip-tip height; terminal slope.	The lip curls down rather than ending as a hill.
Span localization	Row-wise height profile across y .	Center strong, sides faded; no swept tunnel.
Morph continuity	Residual between adjacent morph samples.	No sudden jumps or topology flips.
Slider isolation	Aligned geometries across position/steer/width changes.	Rigid controls stay rigid; width stays span-only.
Visual side check	Browser side view screenshot or direct render inspection.	Curl visible without cross-section mode.

Table 3. Verification gates. A build passing TypeScript is not enough; WaveVis must pass the rendered outcome Alan actually uses.

11 Failure modes

164

The most important failure classes are now explicit:

165

1. **Swept barrel.** One side profile is dragged sideways at constant cross-section. The result reads
as a tunnel, roof, half-pipe, or bridge. 166 167
2. **Side-wall cover.** The lip may exist in a cross-section, but full side view shows a wall covering
the tube. 168 169
3. **Straight chute.** The crest drops nearly vertically to a flat shelf instead of curling forward and
under. 170 171
4. **Blob or lump.** Excess smoothing or broad support creates a mound with no wave identity. 172
5. **Top dent.** The outer crest has a kink or concave dent where a smooth radius is required. 173
6. **Cavity or bowl.** The surface creates an enclosed hollow or bowl-like depression rather than a
plunging sheet. 174 175
7. **Zig-zag linkage.** Neighboring legs alternate direction or length erratically. 176
8. **Hidden mechanism.** Broken topology is ghosted, clipped, or replaced by a cosmetic surface. 177

The solution direction is to simplify the target field while strengthening the gates: localized smooth
wave first, full array second, mechanism-clean render third. Complexity should be added only if it
improves one of those outcomes without breaking the others. 178 179 180

12 Materials and Methods

181

The implementation source is the WaveVis React and TypeScript app (2). The main geometry
source is `src/latticeGeometry.ts`. The primary render source is `src/LatticeViewer3D.tsx`.
Controls are declared in `src/TargetShapeControls.tsx`. Regression checks are run through
`scripts/check-inverse-sheet.cjs`. 182 183 184 185

The public app serves a static build. The architecture PDF is placed in the public proofs directory
as `wavevis-system-architecture.pdf` and linked from the WaveVis control headers. The old 186 187

connector proof remains a source artifact for the mechanism decision, but the visible paper action now points to this architecture record because the current work is broader than connector assignment.

13 Discussion

The main design lesson is that WaveVis cannot be corrected by tuning only the side profile. The user-facing failure was three-dimensional: side walls, swept tubes, missing full-array linkages, broken side/cross-section semantics, and zig-zag legs. The architecture therefore needs a two-level contract. The profile editor controls the desired silhouette. The sheet generator converts that silhouette into a localized displacement field with lateral fade, boundary preservation, and topology completeness.

This distinction also clarifies future work. If the simulator produces a good cross-section but a bad side view, the cross-section is only evidence that the profile exists somewhere. The real artifact still fails if the full three-dimensional render hides the curl or reads as an extrusion. Similarly, if the mesh is clean but the wave looks like a low mound, the mechanism is not enough. Both the shape and the linkage must be correct.

14 Acknowledgements

This record was written to reduce repeated handoff burden during WaveVis development. The repeated complaints about side walls, missing cells, zig-zag legs, cross-section confusion, and swept-barrel geometry are treated as acceptance criteria rather than optional polish notes.

15 Funding

No external funding is claimed in this system architecture record.

16 Author contributions

Alan N. Pham defined the target artifact, accepted and rejected visual states, and the mechanism constraints. Codex generated this architecture record from the current WaveVis source and the active development contract.

17 Competing interests

The author declares no competing interests for this internal architecture record.

212

18 Data availability

213

The source data for this record is the WaveVis source tree, the Alan-supplied Moana ocean reference images copied into the proof figure set, the local proof diagrams, and the current simulator behavior.

214

215

The public PDF is served with the WaveVis static app.

216

19 Code availability

217

The relevant implementation files are `src/latticeGeometry.ts`, `src/LatticeViewer3D.tsx`, `src/TargetShapeControls.tsx`, and `scripts/check-inverse-sheet.cjs` in the WaveVis app repository.

218

219

220

20 Additional information

221

This document supersedes the old header link to the narrow connector-contact constraint proof as the primary public WaveVis PDF. The connector proof remains an implementation reference for mechanism feasibility, but the current public artifact needs the broader system architecture and outcome-verification contract.

222

223

224

225

21 References

226

-
1. A. N. Pham, *WaveVis overhang connector assignment: evidence rejecting all-four-equal X-cells*, Internal WaveVis constraint proof and simulator evidence figures, 2026.
 2. A. N. Pham, *WaveVis inverse-sheet simulator source tree*, Local source files: `src/latticeGeometry.ts`, `src/LatticeViewer3D.tsx`, `src/TargetShapeControls.tsx`, `scripts/check-inverse-sheet.cjs`, 2026.

227

228

229

230